

Manual de Programador – Backend

Proyecto: Backend GymBrisas

Versión: 1.0

Desarrollado en: .NET Core 8 Web API + SQL Server 2022 (Somee)

1. Introducción

El presente documento describe el funcionamiento interno, arquitectura y estructura del backend de la aplicación GymBrisas, desarrollado en ASP.NET Core 8 Web API.

Su propósito es servir como guía técnica para programadores, permitiendo comprender cómo está estructurado el proyecto, cómo interactúa con la base de datos y cómo mantener o extender sus funcionalidades.

2. Objetivo del Sistema

El backend está diseñado exclusivamente para consumir Stored Procedures (SP) ya existentes en la base de datos db_GymBrisas.

Cada endpoint expone una API REST que recibe información desde el frontend o aplicaciones externas y ejecuta el SP correspondiente, retornando los resultados como JSON.

Objetivos específicos:

- Centralizar la comunicación con la base de datos.
- Gestionar usuarios, membresías, métodos de pago y asistencias.
- Automatizar el envío de carnets en PDF a nuevos usuarios.
- Ofrecer endpoints seguros, escalables y fáciles de mantener.

3. Arquitectura del Sistema

El proyecto sigue una arquitectura por capas (MVC simplificada), adaptada para Web API.

Estructura general:

Controllers/ -> Controladores API que ejecutan los SP de la BD.

Funciones/ -> Clases auxiliares (envío de correos, generación de PDF, etc.).

Models/ -> Entidades, DTOs y DbContext.

Recursos/

└─ imgs/ -> Imágenes usadas en carnets y reportes.

└─ pdfs/ -> PDFs generados para envío por correo.

Reportes/ -> Lógica de generación de reportes con QuestPDF.
ViewModels/ -> Modelos de vista usados para estructuras personalizadas.

4. Tecnologías y Herramientas

Lenguaje: C#
Framework: ASP.NET Core 8 Web API
ORM: Entity Framework Core (solo para conexión)
Base de Datos: SQL Server 2022 (Somee Cloud)
Motor de Reportes: QuestPDF
Servidor SMTP: MailKit / System.Net.Mail
IDE: Visual Studio 2022
Hosting: Somee.com

5. Estructura del Proyecto

Controllers: Contiene los controladores donde se ejecutan los SP.
Funciones: Clases auxiliares como GenerarCarnet y EnviarCorreo.
Models: Entidades de base de datos y DbContext.
Recursos: Archivos estáticos (imgs, pdfs).
Reportes: Código para generación de reportes con QuestPDF.
ViewModels: Modelos de vista personalizados.

6. Ejemplo de Endpoint

```
[HttpPut("MetodosPagoActualizar/{id}")]
public async Task<IActionResult> PutMetodos_Pago(int id, Metodos_Pago metodos_Pago)
{
    try
    {
        string connectionString = _context.Database.GetDbConnection().ConnectionString;
        string jsonResult;

        using (var conn = new SqlConnection(connectionString))
        {
            await conn.OpenAsync();
            using (var cmd = new SqlCommand("SP_Metodos_Pagos_Actualizar", conn))
            {
                cmd.CommandType = CommandType.StoredProcedure;
                cmd.Parameters.Add(new SqlParameter("@Id_Metodo", id));
                cmd.Parameters.Add(new SqlParameter("@Descripcion",
metodos_Pago.Descripcion));
```

```

        jsonResult = (await cmd.ExecuteScalarAsync())?.ToString();
        if (string.IsNullOrEmpty(jsonResult))
            return BadRequest(new { success = false, message = "No se recibió respuesta del
SP." });
    }
}

return Content(jsonResult, "application/json");
}
catch (Exception ex)
{
    return BadRequest(new { success = false, message = "Error al actualizar el método de
pago. " + ex.Message });
}
}

```

7. Base de Datos

Motor: SQL Server 2022 (Somee Cloud)

Nombre: db_GymBrisas

Cadena de conexión:

workstation id=db_GymBrisas.mssql.somee.com;

packet size=4096;

user id=Ricardo802_SQLLogin_1;

pwd=jud5snrpzl;

data source=db_GymBrisas.mssql.somee.com;

persist security info=False;

initial catalog=db_GymBrisas;

TrustServerCertificate=True;

8. API REST – Endpoints Principales

Métodos de Pago: PUT /api/MetodosPagoActualizar/{id} -> SP_Metodos_Pagos_Actualizar

Órdenes: POST /api/OrdenesCrear -> SP_Ordenes_Crear

Usuarios: POST /api/UsuariosRegistrar -> SP_Usuarios_Registrar

Rutinas: GET /api/Rutinas/{idUsuario} -> SP_Rutinas_ListarPorUsuario

Membresías: GET /api/Membresias/{id} -> SP_Membresias_Verificar

9. Lógica de Negocio

Toda la lógica reside en los Stored Procedures.

El API recibe parámetros, ejecuta el SP y devuelve la respuesta JSON.

Incluye:

- Verificación de vigencia de membresía (Fn_EstadoMembresia)
- Registro de pagos y membresías (SP_Ordenes_Crear)
- Generación de carnets y envío por correo

10. Seguridad

- Protección de la cadena de conexión en appsettings.json
- Validación básica de errores
- Configuración de CORS
- Posible integración futura con JWT

11. Despliegue

1. Publicar el proyecto (.NET Publish)
2. Subir los archivos a Somee
3. Configurar la conexión en appsettings.json
4. Probar endpoints base

12. Mantenimiento y Extensión

Agregar nuevo endpoint:

1. Crear método en controlador
2. Llamar SP con SqlCommand
3. Retornar JSON

Agregar helper:

1. Crear clase en Funciones/
2. Llamar desde el controlador correspondiente

13. Anexos

- Scripts SQL de SP y funciones.
- Ejemplos JSON.
- Capturas de reportes PDF y carnets.
- Diagrama ER de la base de datos.